```
CLIENTES(id_cliente, nombre, ciudad, saldo)
PEDIDOS(id_pedido, id_cliente, fecha, total)
PRODUCTOS(id_producto, nombre, precio, stock)
DETALLE_PEDIDO(id_detalle, id_pedido, id_producto, cantidad)

INSERT INTO CLIENTES VALUES (1, 'Ana López', 'Sevilla', 1200);
INSERT INTO CLIENTES VALUES (2, 'Juan Pérez', 'Madrid', 500);
INSERT INTO CLIENTES VALUES (3, 'María García', 'Valencia', 2000);
INSERT INTO CLIENTES VALUES (4, 'Carlos Ruiz', 'Sevilla', 300);
INSERT INTO CLIENTES VALUES (5, 'Lucía Fernández', 'Bilbao', 800);

INSERT INTO PRODUCTOS VALUES (1, 'Portátil', 800, 10);
INSERT INTO PRODUCTOS VALUES (2, 'Ratón', 20, 50);
INSERT INTO PRODUCTOS VALUES (3, 'Teclado', 50, 30);
INSERT INTO PRODUCTOS VALUES (4, 'Monitor', 200, 15);
INSERT INTO PRODUCTOS VALUES (5, 'Impresora', 150, 8);

INSERT INTO PEDIDOS VALUES (1, 1, SYSDATE, 0);
INSERT INTO PEDIDOS VALUES (2, 2, SYSDATE, 0);
INSERT INTO PEDIDOS VALUES (3, 3, SYSDATE, 0);
INSERT INTO PEDIDOS VALUES (4, 1, SYSDATE, 0);
INSERT INTO PEDIDOS VALUES (5, 4, SYSDATE, 0);

INSERT INTO DETALLE_PEDIDO VALUES (1, 1, 1, 1); -- portátil
INSERT INTO DETALLE_PEDIDO VALUES (2, 1, 2, 2); -- ratones

INSERT INTO DETALLE_PEDIDO VALUES (3, 2, 3, 1); -- teclado
INSERT INTO DETALLE_PEDIDO VALUES (4, 2, 4, 1); -- monitor

INSERT INTO DETALLE_PEDIDO VALUES (5, 3, 5, 1); -- impresora

CREATE TABLE AUDITORIA (
id_cliente NUMBER,
fecha DATE,
saldo_anterior NUMBER,
saldo_nuevo NUMBER
);

INSERT INTO AUDITORIA VALUES (1, SYSDATE, 1000, 1200);
INSERT INTO AUDITORIA VALUES (2, SYSDATE, 600, 500);
INSERT INTO AUDITORIA VALUES (3, SYSDATE, 1800, 2000);
INSERT INTO AUDITORIA VALUES (4, SYSDATE, 400, 300);
INSERT INTO AUDITORIA VALUES (5, SYSDATE, 700, 800);
```

Ejercicio 1: Función saldo cliente

Se desea implementar una función que reciba el identificador de un cliente y devuelva su saldo actual.
Si el cliente no existe, deberá devolver `NULL`.

Ejercicio 2: Procedimiento actualizar saldo

Crear un procedimiento que permita actualizar el saldo de un cliente sumando o restando una cantidad.

Ejercicio 3: Función contar pedidos

Crear una función que devuelva el número total de pedidos realizados por un cliente.

Ejercicio 4: Insertar pedido

Crear un procedimiento que inserte un nuevo pedido con fecha actual y total inicial 0.

Ejercicio 5: Total gastado

Crear una función que calcule el total gastado por un cliente sumando todos sus pedidos.

Ejercicio 6: Actualizar stock

Crear un procedimiento que reduzca el stock de un producto.
Si no hay suficiente stock, debe mostrar un error.

Ejercicio 7: Comprobar disponibilidad

Crear una función que devuelva `TRUE` si hay suficiente stock de un producto.

Ejercicio 8: Registrar detalle pedido

Crear un procedimiento que:

- Inserte un detalle de pedido
- Actualice el stock del producto

Ejercicio 9: Total de pedido

Crear una función que calcule el total de un pedido sumando cantidad × precio.

Ejercicio 10: Procedimiento completo

Crear un procedimiento que:

- Inserte pedido
- Inserte detalles
- Verifique stock
- Actualice stock
- Calcule total
- Use control de errores con rollback

EJERCICIO 11: Control de crédito de cliente

En una empresa, cada cliente tiene un saldo que representa su crédito disponible.
Se quiere crear un procedimiento que:

1. Reciba un `id_cliente` y un importe de compra
2. Verifique si el cliente tiene saldo suficiente
3. Si tiene:
   - Reste el saldo
   - Permita la operación
4. Si no tiene:
   - Lance un error personalizado

☞ **No vale solo un UPDATE**, porque hay lógica condicional + control de errores.

EJERCICIO 12: Aplicar descuentos progresivos

Crear una función que calcule el precio final de un pedido según reglas:

- 100€ → 10% descuento
- 500€ → 20% descuento
- Cliente VIP (saldo > 1000) → +5% extra

☞ Esto **no es un simple SELECT**, hay reglas encadenadas.

EJERCICIO 13: Procesar pedidos en lote (uso de cursor)

Crear un procedimiento que recorra todos los pedidos sin total (`total = 0`) y:

1. Calcule su total
2. Actualice la tabla
3. Muestre por pantalla cada actualización

☞ Aquí **sí necesitas recorrer filas (programación real)**.

EJERCICIO 14: Registro de auditoría

Crear un procedimiento que:

1. Actualice el saldo de un cliente
2. Guarde un registro en una tabla `AUDITORIA` con:
   - id_cliente
   - fecha
   - saldo anterior
   - saldo nuevo

☞ SQL solo no guarda "histórico + lógica".

EJERCICIO 15: Simulación de sistema completo (transacción compleja)

Crear un procedimiento que simule una compra completa:

1. Verifica stock de TODOS los productos del pedido
2. Si alguno falla → rollback total
3. Si todo correcto:
    - Actualiza stock
    - Calcula total
    - Aplica descuento
    - Actualiza pedido

☞ Esto es **programación real (flujo + control + rollback)**.